Иу5-23М Лавренов РК1

Группа: ИУ5-23М **Вариант:** 5

Задачи по варианту

Задача №5. Для набора данных провести кодирование одного (произвольного) категориального признака с использованием метода **one-hot encoding**.

Задача №25. Для набора данных для одного (произвольного) числового признака провести **обнаружение и удаление выбросов на основе межквартильного размаха (IQR)**.

Дополнительное задание для группы ИУ5-23М. Для произвольной колонки данных построить график **«Ящик с усами» (boxplot)**.

1. Импорт библиотек

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (10, 6)

print('numpy  :', np.__version__)
print('pandas :', pd.__version__)
print('seaborn:', sns.__version__)
```

```
numpy  : 2.0.2
pandas : 2.2.2
seaborn: 0.13.2
```

2. Загрузка набора данных

Датасет Titanic загружается через `seaborn.load_dataset('titanic')`. На случай, если seaborn не сможет скачать данные, предусмотрен резервный вариант — загрузка CSV напрямую с GitHub.

In [2]:
```
try:
    df = sns.load_dataset('titanic')
    print('Источник: seaborn')
except Exception as e:
```

```
    print('seaborn недоступен, пробуем загрузить с GitHub:', e)
    URL = 'https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tit
    df = pd.read_csv(URL)
    print('Источник: GitHub')

print('Размер датасета:', df.shape)
df.head()
```

```
Источник: seaborn
Размер датасета: (891, 15)
```

Out[2]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	ac
0	0	3	male	22.0	1	0	7.2500	S	Third	man	
1	1	1	female	38.0	1	0	71.2833	C	First	woman	
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	
3	1	1	female	35.0	1	0	53.1000	S	First	woman	
4	0	3	male	35.0	0	0	8.0500	S	Third	man	

In [3]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

In [4]:

```
df.describe(include='all').T
```

Out[4]:

	count	unique	top	freq	mean	std	min	25%
survived	891.0	NaN	NaN	NaN	0.383838	0.486592	0.0	0.0
pclass	891.0	NaN	NaN	NaN	2.308642	0.836071	1.0	2.0
sex	891	2	male	577	NaN	NaN	NaN	NaN
age	714.0	NaN	NaN	NaN	29.699118	14.526497	0.42	20.125
sibsp	891.0	NaN	NaN	NaN	0.523008	1.102743	0.0	0.0
parch	891.0	NaN	NaN	NaN	0.381594	0.806057	0.0	0.0
fare	891.0	NaN	NaN	NaN	32.204208	49.693429	0.0	7.9104
embarked	889	3	S	644	NaN	NaN	NaN	NaN
class	891	3	Third	491	NaN	NaN	NaN	NaN
who	891	3	man	537	NaN	NaN	NaN	NaN
adult_male	891	2	True	537	NaN	NaN	NaN	NaN
deck	203	7	C	59	NaN	NaN	NaN	NaN
embark_town	889	3	Southampton	644	NaN	NaN	NaN	NaN
alive	891	2	no	549	NaN	NaN	NaN	NaN
alone	891	2	True	537	NaN	NaN	NaN	NaN

3. Задача №5 — One-hot encoding категориального признака

В качестве кодируемого признака выбран `embarked` — порт посадки пассажира. Он принимает три значения:

- `S` — Southampton
- `C` — Cherbourg
- `Q` — Queenstown

Это типичный номинальный признак (между значениями нет естественного порядка), поэтому one-hot encoding для него подходит лучше всего.

3.1. Анализ признака до кодирования

In [5]:

```
print('Уникальные значения признака embarked:')
print(df['embarked'].value_counts(dropna=False))
print('\nДоля пропусков:', round(df['embarked'].isna().mean(), 4))
```

```
Уникальные значения признака embarked:
embarked
S      644
C      168
Q       77
NaN      2
Name: count, dtype: int64

Доля пропусков: 0.0022
```

In [6]:
```python
# В признаке есть пропуски — заполним модой (самым частым значением),
# чтобы при кодировании не появлялся отдельный столбец для NaN.
mode_value = df['embarked'].mode()[0]
print(f'Мода признака embarked: {mode_value!r}')

df['embarked'] = df['embarked'].fillna(mode_value)
print('Пропусков после заполнения:', df['embarked'].isna().sum())
```

```
Мода признака embarked: 'S'
Пропусков после заполнения: 0
```

3.2. Применение one-hot encoding

Используем функцию `pandas.get_dummies`. Она создаёт по одному бинарному столбцу для каждой категории. Параметр `prefix='embarked'` задаёт префикс имён новых столбцов, а `dtype=int` — числовой тип (0/1) вместо булевого.

In [7]:
```python
embarked_ohe = pd.get_dummies(df['embarked'], prefix='embarked', dtype=int)
embarked_ohe.head(10)
```

Out[7]:

	embarked_C	embarked_Q	embarked_S
0	0	0	1
1	1	0	0
2	0	0	1
3	0	0	1
4	0	0	1
5	0	1	0
6	0	0	1
7	0	0	1
8	0	0	1
9	1	0	0

In [8]:
```python
# Присоединим закодированные столбцы к датафрейму, удалив исходный признак
df_encoded = pd.concat([df.drop(columns=['embarked']), embarked_ohe], axis=1

print('Размер до кодирования  :', df.shape)
```

```
print('Размер после кодирования:', df_encoded.shape)
df_encoded.head()
```

```
Размер до кодирования  : (891, 15)
Размер после кодирования: (891, 17)
```

Out[8]:

	survived	pclass	sex	age	sibsp	parch	fare	class	who	adult_male	c
0	0	3	male	22.0	1	0	7.2500	Third	man	True	
1	1	1	female	38.0	1	0	71.2833	First	woman	False	
2	1	3	female	26.0	0	0	7.9250	Third	woman	False	
3	1	1	female	35.0	1	0	53.1000	First	woman	False	
4	0	3	male	35.0	0	0	8.0500	Third	man	True	

3.3. Замечание про «dummy variable trap»

При построении линейных моделей часто используют параметр `drop_first=True`, чтобы убрать один из столбцов и избежать линейной зависимости между признаками (мультиколлинеарности). Покажем и такой вариант:

In [9]:
```
embarked_ohe_drop = pd.get_dummies(df['embarked'], prefix='embarked', dtype=
embarked_ohe_drop.head()
```

Out[9]:

	embarked_Q	embarked_S
0	0	1
1	0	0
2	0	1
3	0	1
4	0	1

4. Задача №25 — Обнаружение и удаление выбросов по IQR

В качестве числового признака выбран **`fare`** — стоимость билета. Этот признак известен сильной правой асимметрией и наличием очень крупных значений (билеты первого класса в каютах-люкс), что делает его удобным примером для IQR-фильтрации.

4.1. Метод межквартильного размаха

Алгоритм:

1. Вычисляем квартили: Q_1 (25-й перцентиль) и Q_3 (75-й перцентиль).
2. Считаем межквартильный размах: $\mathrm{IQR} = Q_3 - Q_1$.
3. Определяем границы:
   - нижняя: $Q_1 - 1{,}5 \cdot \mathrm{IQR}$
   - верхняя: $Q_3 + 1{,}5 \cdot \mathrm{IQR}$
4. Все наблюдения вне этих границ считаются выбросами и удаляются.

In [10]:
```
feature = 'fare'

Q1 = df_encoded[feature].quantile(0.25)
Q3 = df_encoded[feature].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

print(f'Q1                = {Q1:.4f}')
print(f'Q3                = {Q3:.4f}')
print(f'IQR               = {IQR:.4f}')
print(f'Нижняя граница    = {lower:.4f}')
print(f'Верхняя граница   = {upper:.4f}')
```

```
Q1                = 7.9104
Q3                = 31.0000
IQR               = 23.0896
Нижняя граница    = -26.7240
Верхняя граница   = 65.6344
```

In [11]:
```
# Маска выбросов
outlier_mask = (df_encoded[feature] < lower) | (df_encoded[feature] > upper)
n_outliers = int(outlier_mask.sum())

print(f'Количество выбросов в признаке {feature!r}: {n_outliers}')
print(f'Доля выбросов: {n_outliers / len(df_encoded):.2%}')

# Покажем самые крупные выбросы
df_encoded.loc[outlier_mask, [feature]].sort_values(feature, ascending=False
```

```
Количество выбросов в признаке 'fare': 116
Доля выбросов: 13.02%
```

Out[11]:

	fare
258	512.3292
737	512.3292
679	512.3292
27	263.0000
341	263.0000
438	263.0000
88	263.0000
311	262.3750
742	262.3750
299	247.5208

In [12]:

```
# Удаляем строки-выбросы
df_clean = df_encoded.loc[~outlier_mask].copy()

print('Размер до удаления выбросов :', df_encoded.shape)
print('Размер после удаления       :', df_clean.shape)
```

```
Размер до удаления выбросов : (891, 17)
Размер после удаления       : (775, 17)
```

4.2. Сравнение распределения признака до и после фильтрации

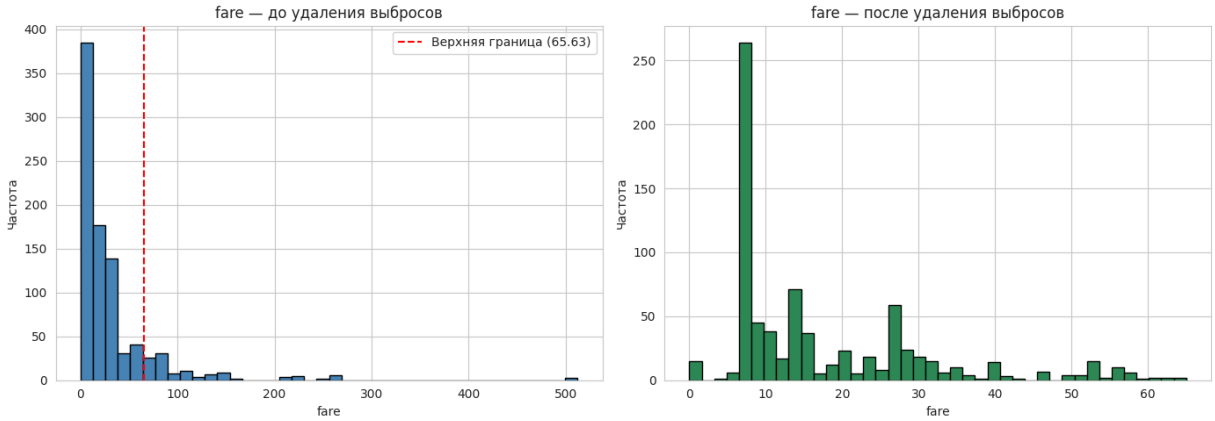
In [13]:

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(df_encoded[feature], bins=40, color='steelblue', edgecolor='bla
axes[0].axvline(upper, color='red', linestyle='--', label=f'Верхняя граница
axes[0].set_title(f'{feature} — до удаления выбросов')
axes[0].set_xlabel(feature)
axes[0].set_ylabel('Частота')
axes[0].legend()

axes[1].hist(df_clean[feature], bins=40, color='seagreen', edgecolor='black'
axes[1].set_title(f'{feature} — после удаления выбросов')
axes[1].set_xlabel(feature)
axes[1].set_ylabel('Частота')

plt.tight_layout()
plt.show()
```



In [14]:
```
# Сводная статистика до и после удаления выбросов
pd.concat(
    [df_encoded[feature].describe().rename('до'),
     df_clean[feature].describe().rename('после')],
    axis=1
)
```

Out[14]:

	до	после
count	891.000000	775.000000
mean	32.204208	17.822091
std	49.693429	13.578085
min	0.000000	0.000000
25%	7.910400	7.895800
50%	14.454200	13.000000
75%	31.000000	26.000000
max	512.329200	65.000000

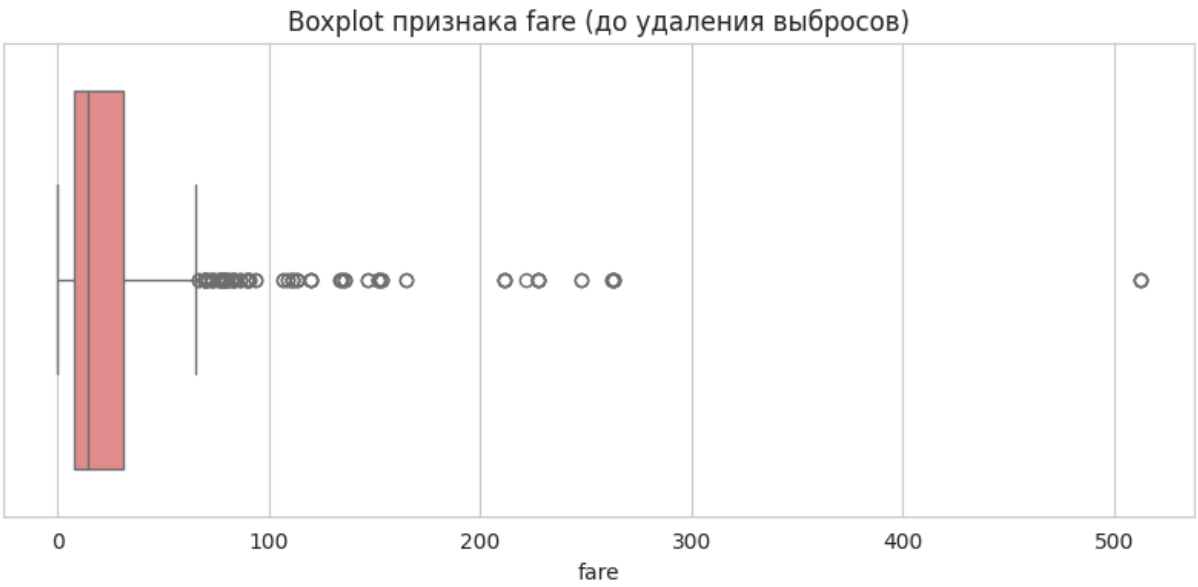
5. Дополнительное задание — Boxplot («ящик с усами»)

Построим boxplot для признака `fare`. Это особенно показательно, потому что именно по логике boxplot работает и метод IQR: «ящик» — это межквартильный размах, «усы» — границы $Q_1 - 1{,}5 \cdot \mathrm{IQR}$ и $Q_3 + 1{,}5 \cdot \mathrm{IQR}$, а точки за усами — те самые выбросы.

5.1. Boxplot для всего признака `fare` (до удаления выбросов)

In [15]:
```
plt.figure(figsize=(10, 4))
sns.boxplot(x=df_encoded[feature], color='lightcoral')
plt.title(f'Boxplot признака {feature} (до удаления выбросов)')
```

```
plt.xlabel(feature)
plt.show()
```



На графике хорошо видно множество точек справа — это и есть выбросы по IQR-критерию.

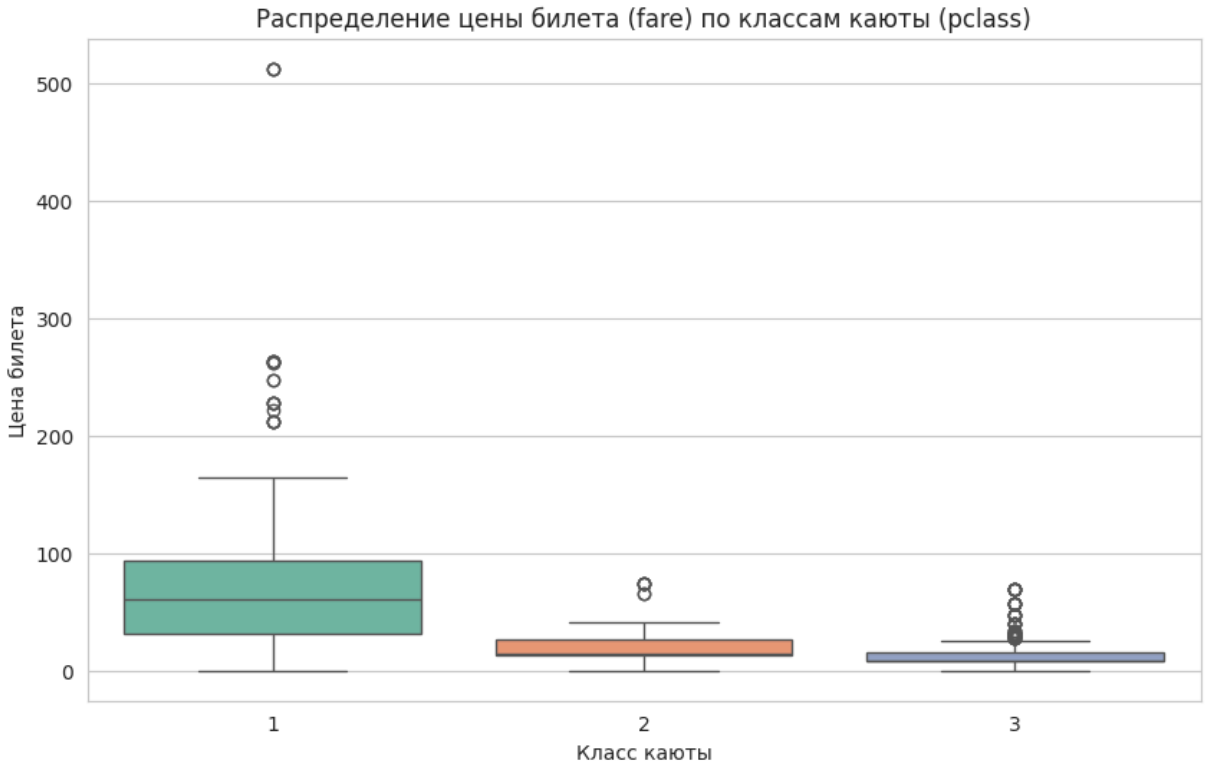
5.2. Boxplot после удаления выбросов

In [16]:
```
plt.figure(figsize=(10, 4))
sns.boxplot(x=df_clean[feature], color='lightgreen')
plt.title(f'Boxplot признака {feature} (после удаления выбросов)')
plt.xlabel(feature)
plt.show()
```

5.3. Boxplot признака `fare` в разрезе класса каюты `pclass`

Дополнительно построим boxplot с группировкой — это типичный сценарий использования «ящика с усами» для сравнения распределений между группами.

In [17]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(x='pclass', y='fare', data=df, hue='pclass', palette='Set2', leg
plt.title('Распределение цены билета (fare) по классам каюты (pclass)')
plt.xlabel('Класс каюты')
plt.ylabel('Цена билета')
plt.show()
```



6. Выводы

1. **Задача №5 (one-hot encoding).** Категориальный признак `embarked` с тремя значениями (`S` , `C` , `Q`) был закодирован методом one-hot encoding с помощью `pandas.get_dummies` . Получены три бинарных столбца `embarked_C` , `embarked_Q` , `embarked_S` . Отдельно показан вариант с `drop_first=True` , который применяют для линейных моделей во избежание мультиколлинеарности.

2. **Задача №25 (IQR-фильтрация).** Для признака `fare` вычислены Q_1, Q_3 и IQR, после чего определены допустимые границы $[Q_1 - 1{,}5 \cdot \mathrm{IQR};\ Q_3 + 1{,}5 \cdot \mathrm{IQR}]$. Найденные выбросы удалены. После фильтрации распределение `fare` стало значительно более симметричным, а характеристики — устойчивее к крайним значениям.

3. **Boxplot.** Графики «ящик с усами» наглядно показали наличие выбросов до фильтрации и их отсутствие после, а также подтвердили закономерное различие распределения цены билета между классами каюты (медианная цена в первом классе значительно выше).